arXiv:2412.06769v3 [cs.CL] 3 Nov 2025

Training Large Language Models to Reason in a Continuous Latent Space

Shibo Hao[1,2,*], **Sainbayar Sukhbaatar**[1], **DiJia Su**[1], **Xian Li**[1], **Zhiting Hu**[2], **Jason Weston**[1], **Yuandong Tian**[1]

[1]FAIR at Meta, [2]UC San Diego
[*]Work done at Meta

Large language models (LLMs) are restricted to reason in the "language space", where they typically express the reasoning process with a chain-of-thought (CoT) to solve a complex reasoning problem. However, we argue that language space may not always be optimal for reasoning. For example, most word tokens primarily ensure textual coherence and are not essential for reasoning, while some critical tokens require complex planning and pose huge challenges to LLMs. To explore the potential of LLM reasoning in an unrestricted latent space instead of using natural language, we introduce a new paradigm COCONUT (Chain of Continuous Thought). We utilize the last hidden state of the LLM as a representation of the reasoning state (termed "continuous thought"). Rather than decoding this into a word token, we feed it back to the LLM as the subsequent input embedding directly in the continuous space. This latent reasoning paradigm leads to the emergence of an advanced reasoning pattern: the continuous thought can encode multiple alternative next reasoning steps, allowing the model to perform a breadth-first search (BFS) to solve the problem, rather than prematurely committing to a single deterministic path like CoT. COCONUT outperforms CoT on certain logical reasoning tasks that require substantial search during planning, and shows a better trade-off between accuracy and efficiency.

Last updated: November 4, 2025
Code: https://github.com/facebookresearch/coconut

Meta

1 Introduction

Large language models (LLMs) have demonstrated remarkable reasoning abilities, emerging from extensive pretraining on human languages (Dubey et al., 2024; Achiam et al., 2023). While next token prediction is an effective training objective, it imposes a fundamental constraint on the LLM as a reasoning machine: the explicit reasoning process of LLMs must be generated in word tokens. For example, a prevalent approach, known as chain-of-thought (CoT) reasoning (Wei et al., 2022), involves prompting or training LLMs to generate solutions step-by-step using natural language. However, this is in stark contrast to certain human cognition results. Neuroimaging studies have consistently shown that the language network – a set of brain regions responsible for language comprehension and production – remains largely inactive during various reasoning tasks (Amalric and Dehaene, 2019; Monti et al., 2012, 2007, 2009; Fedorenko et al., 2011). Further evidence indicates that human language is optimized for communication rather than reasoning (Fedorenko et al., 2024).

A significant issue arises when LLMs use language for reasoning: the amount of reasoning required for each particular token varies greatly, yet current LLM architectures allocate nearly the same computing budget for predicting every token. Most tokens in a reasoning chain are generated solely for fluency, contributing little to the actual reasoning process. By contrast, some critical tokens require complex planning and pose huge challenges to LLMs. While previous work has attempted to fix these problems by prompting LLMs to generate succinct reasoning chains (Madaan and Yazdanbakhsh, 2022), or performing additional reasoning before generating some critical tokens (Zelikman et al., 2024), these solutions remain constrained within the language space and do not solve the fundamental problems. On the contrary, it would be ideal for LLMs to have the freedom to reason without any language constraints, and then translate their findings into language only when necessary.

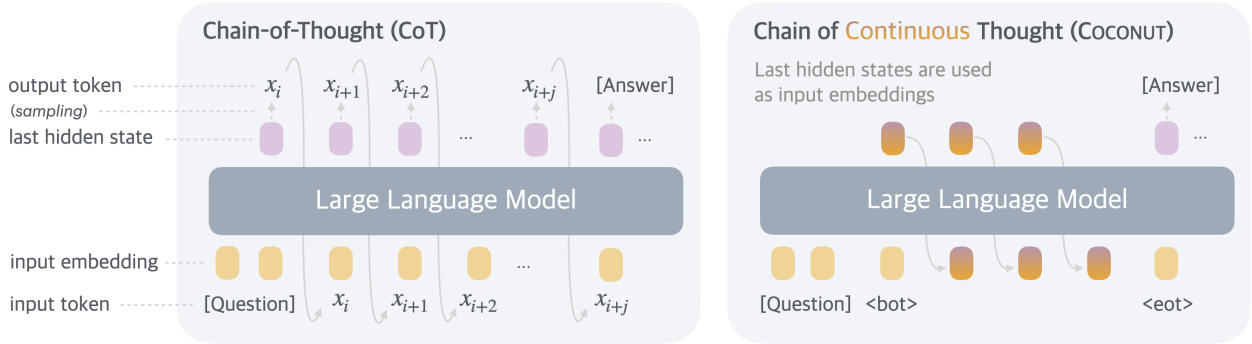

Figure 1 A comparison of Chain of Continuous Thought (Coconut) with Chain-of-Thought (CoT). In CoT, the model generates the reasoning process as a word token sequence (e.g., $[x_i, x_{i+1}, ..., x_{i+j}]$ in the figure). Coconut regards the last hidden state as a representation of the reasoning state (termed "continuous thought"), and directly uses it as the next input embedding. This allows the LLM to reason in an unrestricted latent space instead of a language space.

In this work we instead explore LLM reasoning in a latent space by introducing a novel paradigm, Coconut (Chain of Continuous Thought). It involves a simple modification to the traditional CoT process: instead of mapping between hidden states and language tokens using the language model head and embedding layer, Coconut directly feeds the last hidden state (a continuous thought) as the input embedding for the next token (Figure 1). This modification frees the reasoning from being within the language space, and the system can be optimized end-to-end by gradient descent, as continuous thoughts are fully differentiable. To enhance the training of latent reasoning, we employ a multi-stage training strategy inspired by Deng et al. (2024), which effectively utilizes language reasoning chains to guide the training process.

Interestingly, our proposed paradigm leads to an efficient reasoning pattern. Unlike language-based reasoning, continuous thoughts in Coconut can encode multiple potential next steps simultaneously, allowing for a reasoning process akin to breadth-first search (BFS). While the model may not initially make the correct decision, it can maintain many possible options within the continuous thoughts and progressively eliminate incorrect paths through reasoning, guided by some implicit value functions. This advanced reasoning mechanism surpasses traditional CoT, even though the model is not explicitly trained or instructed to operate in this manner, as seen in previous works (Yao et al., 2023; Hao et al., 2023).

Experimentally, Coconut successfully enhances the reasoning capabilities of LLMs. For math reasoning (GSM8k, Cobbe et al., 2021), using continuous thoughts is shown to be beneficial to reasoning accuracy, mirroring the effects of language reasoning chains. This indicates the potential to scale and solve increasingly challenging problems by chaining more continuous thoughts. On logical reasoning including ProntoQA (Saparov and He, 2022), and our newly proposed ProsQA (Section 4) which requires stronger planning ability, Coconut and some of its variants even surpasses language-based CoT methods, while generating significantly fewer tokens during inference. We believe that these findings underscore the potential of latent reasoning and could provide valuable insights for future research.

2 Related Work

Chain-of-thought (CoT) reasoning. We use the term chain-of-thought broadly to refer to methods that generate an intermediate reasoning process in language before outputting the final answer. This includes prompting LLMs (Wei et al., 2022; Khot et al., 2022; Zhou et al., 2022), or training LLMs to generate reasoning chains, either with supervised finetuning (Yue et al., 2023; Yu et al., 2023) or reinforcement learning (Wang et al., 2024; Havrilla et al., 2024; Shao et al., 2024; Yu et al., 2024a). Madaan and Yazdanbakhsh (2022) classified the tokens in CoT into symbols, patterns, and text, and proposed to guide the LLM to generate concise CoT based on analysis of their roles. Recent theoretical analyses have demonstrated the usefulness of CoT from the perspective of model expressivity (Feng et al., 2023; Merrill and Sabharwal, 2023; Li et al., 2024). By employing CoT, the effective depth of the transformer increases because the generated outputs are looped back to the input (Feng et al., 2023). These analyses, combined with the established effectiveness of CoT,

motivated our design of feeding the continuous thoughts back into the LLM as input embeddings. While CoT has proven effective for certain tasks, its autoregressive generation nature makes it challenging to mimic human reasoning on more complex problems (LeCun, 2022; Hao et al., 2023), which typically require planning and search. There are works that equip LLMs with explicit tree search algorithms (Xie et al., 2023; Yao et al., 2023; Hao et al., 2024), or train the LLM on search dynamics and trajectories (Lehnert et al., 2024; Gandhi et al., 2024; Su et al., 2024). In our analysis, we find that after removing the constraint of a language space, a new reasoning pattern similar to BFS emerges, even though the model is not explicitly trained in this way.

Latent reasoning in LLMs. Previous works mostly define latent reasoning in LLMs as the hidden computation in transformers (Yang et al., 2024; Biran et al., 2024). Yang et al. (2024) constructed a dataset of two-hop reasoning problems and discovered that it is possible to recover the intermediate variable from the hidden representations. Biran et al. (2024) further proposed to intervene the latent reasoning by "back-patching" the hidden representation. Shalev et al. (2024) discovered parallel latent reasoning paths in LLMs. Another line of work has discovered that, even if the model generates a CoT to reason, the model may actually utilize a different latent reasoning process. This phenomenon is known as the unfaithfulness of CoT reasoning (Wang et al., 2022; Turpin et al., 2024). To enhance the latent reasoning of LLMs, previous research proposed to augment it with additional tokens. Goyal et al. (2023) pretrained the model by randomly inserting a learnable `<pause>` tokens to the training corpus. This improves LLM's performance on a variety of tasks, especially when followed by supervised finetuning with `<pause>` tokens. On the other hand, Pfau et al. (2024) further explored the usage of filler tokens, e.g., "...", and concluded that they work well for highly parallelizable problems. However, Pfau et al. (2024) mentioned these methods do not extend the expressivity of the LLM like CoT; hence, they may not scale to more general and complex reasoning problems. Wang et al. (2023) proposed to predict a planning token as a discrete latent variable before generating the next reasoning step. Recently, it has also been found that one can "internalize" the CoT reasoning into latent reasoning in the transformer with knowledge distillation (Deng et al., 2023) or a special training curriculum which gradually shortens CoT (Deng et al., 2024). Yu et al. (2024b) also proposed to distill a model that can reason latently from data generated with complex reasoning algorithms. These training methods can be combined to our framework, and specifically, we find that breaking down the learning of continuous thoughts into multiple stages, inspired by iCoT (Deng et al., 2024), is very beneficial for the training. Other work explores alternative architectures for latent reasoning, including looped transformers (Giannou et al., 2023; Fan et al., 2024), diffusion models in sentence embedding space (Barrault et al., 2024). Different from these works, we focus on general multi-step reasoning tasks and aim to investigate the unique properties of latent reasoning in comparison to language space. In addition to reasoning tasks, Pham et al. (2023) also explored using continuous space for multi-agent communication. Building on Coconut, Zhu et al. (2025b) developed a theoretical framework demonstrating that continuous CoT can be more efficient than discrete CoT on certain tasks by encoding multiple reasoning paths in superposition states. Subsequently, Zhu et al. (2025a) analyzed the training dynamics to explain how such superposition emerges under the Coconut training objective.

3 Coconut: Chain of Continuous Thought

In this section, we introduce our new paradigm Coconut (Chain of Continuous Thought) for reasoning in an unconstrained latent space. We begin by introducing the background and notation we use for language models. For an input sequence $x = (x_1, ..., x_T)$, the standard large language model $\mathcal{M}$ can be described as:

$$H_t = \text{Transformer}(E_t)$$
$$\mathcal{M}(x_{t+1} \mid x_{\leq t}) = \text{softmax}(W h_t)$$

where $E_t = [e(x_1), e(x_2), ..., e(x_t)]$ is the sequence of token embeddings up to position t; $H_t \in \mathbb{R}^{t \times d}$ is the matrix of the last hidden states for all tokens up to position t; h_t is the last hidden state of position t, i.e., $h_t = H_t[t, :]$; $e(\cdot)$ is the token embedding function; W is the parameter of the language model head.

Method Overview. In the proposed Coconut method, the LLM switches between the "language mode" and "latent mode" (Figure 1). In language mode, the model operates as a standard language model, autoregressively